

System Design Basics

Lesson 2: Key Design Principles

Key Design Principles – Study Notes

Key Concepts

1. **Scalability** – The ability of a system to handle growing workload by adding resources.
2. **Availability** – The proportion of time a system is operational and accessible to users.
3. **Reliability** – The consistency of a system's performance and its ability to avoid failures.
4. **Maintainability** – How easily a system can be repaired, updated, or enhanced.
5. **Trade off Awareness** – Understanding that improving one principle often impacts the others.

Summary

Designing robust software or infrastructure requires balancing four fundamental qualities: scalability, availability, reliability, and maintainability. **Scalability** is like adding more lanes to a highway when traffic increases; a well scaled system can grow horizontally (more nodes) or vertically (bigger machines) without a redesign. **Availability** is akin to a store that stays open most of the time—users expect it to be reachable whenever they need it, which is achieved through redundancy, failover, and careful capacity planning.

Reliability reflects how consistently the “store” operates without unexpected closures or glitches; it is built through fault tolerant architecture, automated testing, and graceful degradation. **Maintainability** is the ease with which you can fix a broken shelf, repaint the walls, or add a new product line—clear code structure, documentation, modular components, and observability all contribute. Mastering these principles helps you create systems that grow smoothly, stay up, behave predictably, and evolve without costly downtime.

Important Points

- **Scalability**
 - **Horizontal** (add more nodes) vs. **vertical** (bigger node).
 - Stateless services and load balancers simplify horizontal scaling.
 - Bottlenecks often appear in shared resources (e.g., single database).
- **Availability**
 - Measured as **Uptime** = (Total Time – Downtime) / Total Time (often expressed as “nines”).
 - Redundancy (active active, active passive) and health checks enable rapid failover.
 - SLA (Service Level Agreement) defines required availability for customers.
- **Reliability**
 - Involves **fault tolerance** (system continues despite component failures).
 - Use patterns like circuit breaker, retry with exponential back off, and graceful degradation.

- Monitoring, logging, and alerting are essential to detect and react to failures quickly.
 - **Maintainability**
- Favor *modular* and *loose coupled* design; each module can be changed independently.
- Write self documenting code, enforce coding standards, and keep documentation up to date.
- Automated CI/CD pipelines reduce manual steps and lower the chance of human error.
 - **Trade offs**
- High availability often requires extra hardware !' higher cost.
- Extreme scalability may introduce complexity that hurts maintainability.
- Strive for a balanced “design triangle” where the three sides (scalability, availability, maintainability) meet the project's priorities.

Examples

1. E commerce Website (Scalability & Availability)

- **Analogy:** A pop up shop that can quickly add more checkout counters when a sale starts.
- **Implementation:**
- Stateless web servers behind a load balancer !' add/remove instances on demand (auto scaling).
- Database read replicas for read heavy traffic; a primary secondary failover for writes.
- Health checks remove unhealthy instances automatically, keeping the site available 99.9% of the time.

2. Microservice for Payment Processing (Reliability & Maintainability)

- **Analogy:** A vending machine that continues dispensing items even if one coin slot jams.
- **Implementation:**
- Circuit breaker pattern prevents cascading failures when the downstream bank API is slow.
- Each payment step is a separate microservice (validation, authorization, settlement) !' isolated changes don't affect the whole flow.
- Comprehensive unit/integration tests and CI pipeline allow rapid, safe deployments, keeping the system reliable and easy to maintain.

Quick Review

1. **What is the primary difference between horizontal and vertical scaling?**
2. **How does a load balancer contribute to both scalability and availability?**
3. **Name two design patterns that improve reliability and briefly describe them.**
4. **Why is modularity important for maintainability?**
5. **Give an example of a trade off you might encounter when increasing availability.**

Further Reading

- **CAP Theorem** – Understanding consistency, availability, and partition tolerance trade offs.
- **Design for Failure** – Patterns like bulkheads, retries, and graceful degradation.
- **Observability** – Metrics, tracing, and logging for monitoring reliability and performance.
- **Infrastructure as Code (IaC)** – Using tools like Terraform or CloudFormation to manage

scalable, repeatable environments.

- **Domain Driven Design (DDD)** – Helps create maintainable, well structured bounded contexts.

